

UNIVERSIDADE FEDERAL DO AMAZONAS
INSTITUTO DE COMPUTAÇÃO
GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

SISTEMAS OPERACIONAIS I
1º TRABALHO PRÁTICO

JOSUÉ BRITO MOURA
KAREN JULIANA BÁEZ GONZÁLEZ
LUNA VEIGA HORTA BRAGA

Manaus – AM

2026

JOSUÉ BRITO MOURA
KAREN JULIANA BÁEZ GONZÁLEZ
LUNA VEIGA HORTA BRAGA

SISTEMAS OPERACIONAIS I
1º TRABALHO PRÁTICO

Relatório do 1º Trabalho Prático da
disciplina de Sistemas Operacionais I,
que consiste na implementação de
soluções para os problemas clássicos de
sincronização Leitores × Escritores e
Produtores × Consumidores

Manaus – AM

2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. PROCESSOS LEITORES X ESCRITORES.....	4
2.1 Leitores e escritores sem preferência de ordem de acesso.....	4
2.2 Escritores com preferência sobre leitores.....	6
2.3 Versão sem controle de concorrência.....	9
3. PROCESSOS PRODUTORES X CONSUMIDORES.....	11
3.1 Vários processos produtores e 1 consumidor.....	12
3.2 Vários processos produtores e consumidores.....	14
3.3 Versão sem controle de concorrência.....	16
4. CONCLUSÃO.....	18
5. FONTES.....	19

1. INTRODUÇÃO

O trabalho tem como objetivo aplicar os conceitos de programação concorrente por meio da implementação de threads e mecanismos de sincronização com semáforos. Cada problema é implementado em três versões, incluindo uma sem controle de concorrência, a fim de demonstrar os problemas que podem ocorrer no acesso simultâneo a dados compartilhados.

A Seção 2 apresenta as implementações do problema Leitores × Escritores, enquanto a Seção 3 aborda o problema Produtores × Consumidores. Em ambas, são descritas as escolhas de implementação e apresentados exemplos de execução.

2. PROCESSOS LEITORES X ESCRITORES

2.1 Leitores e escritores sem preferência de ordem de acesso

O tema escolhido foi um sistema bancário. Os dados compartilhados são duas variáveis: saldo, que começa com 1000, e extrato, que soma as operações já registradas. Quando os dados estão corretos, $\text{saldo} = 1000 + \text{extrato}$. Os leitores conferem essa relação, e os escritores fazem depósitos ou saques.

A quantidade de leitores e escritores, os atrasos, o tempo de escrita e o valor de cada escritor são digitados no teclado, o que permite testar qualquer configuração. Cada thread executa 5 rodadas.

Nesta versão, apenas os escritores são sincronizados, por um semáforo binário (`sem_escritores`), de modo que só um escritor altera os dados por vez. Os leitores não usam bloqueio, e não há preferência entre as duas classes: a ordem de acesso depende apenas do escalonador. O escritor atualiza o extrato, espera o tempo de escrita e só depois atualiza o saldo. Se um leitor ler nesse intervalo, vê os dois valores inconsistentes, o que é a leitura suja, e o programa conta quantas ocorreram.

A saída mostra, com o tempo em milissegundos, quando cada thread é criada, entra e sai da região crítica, é bloqueada e finaliza, além dos valores lidos e gravados.

- Exemplos de Execução

Exemplo 1

```

[ 121ms] ESCRITOR 1 | bloqueado: outro escritor ativo
[ 212ms] ESCRITOR 0 | saldo atualizado: saldo=1050 extrato=50
[ 212ms] ESCRITOR 0 | saiu da região crítica
[ 212ms] ESCRITOR 1 | entrou na região crítica
[ 212ms] ESCRITOR 1 | saque de 50: extrato=0 registrado, saldo=1050 ainda desatualizado
[ 222ms] ESCRITOR 0 | bloqueado: outro escritor ativo
[ 312ms] ESCRITOR 1 | saldo atualizado: saldo=1000 extrato=0
[ 312ms] ESCRITOR 1 | saiu da região crítica
[ 312ms] ESCRITOR 0 | entrou na região crítica
[ 313ms] ESCRITOR 0 | depósito de 100: extrato=100 registrado, saldo=1000 ainda desatualizado
[ 322ms] ESCRITOR 1 | bloqueado: outro escritor ativo
[ 413ms] ESCRITOR 0 | saldo atualizado: saldo=1100 extrato=100
[ 413ms] ESCRITOR 0 | saiu da região crítica
[ 413ms] ESCRITOR 1 | entrou na região crítica
[ 413ms] ESCRITOR 1 | saque de 50: extrato=50 registrado, saldo=1100 ainda desatualizado
[ 424ms] ESCRITOR 0 | bloqueado: outro escritor ativo
[ 514ms] ESCRITOR 1 | saldo atualizado: saldo=1050 extrato=50
[ 514ms] ESCRITOR 1 | saiu da região crítica
[ 514ms] ESCRITOR 0 | entrou na região crítica
[ 515ms] ESCRITOR 0 | depósito de 100: extrato=150 registrado, saldo=1050 ainda desatualizado
[ 525ms] ESCRITOR 1 | bloqueado: outro escritor ativo
[ 615ms] ESCRITOR 0 | saldo atualizado: saldo=1150 extrato=150
[ 615ms] ESCRITOR 0 | saiu da região crítica
[ 615ms] ESCRITOR 1 | entrou na região crítica
[ 615ms] ESCRITOR 1 | saque de 50: extrato=100 registrado, saldo=1150 ainda desatualizado
[ 625ms] ESCRITOR 0 | bloqueado: outro escritor ativo
[ 716ms] ESCRITOR 1 | saldo atualizado: saldo=1100 extrato=100
[ 716ms] ESCRITOR 1 | saiu da região crítica
[ 716ms] ESCRITOR 0 | entrou na região crítica
[ 716ms] ESCRITOR 0 | depósito de 100: extrato=200 registrado, saldo=1100 ainda desatualizado
[ 726ms] ESCRITOR 1 | bloqueado: outro escritor ativo
[ 817ms] ESCRITOR 0 | saldo atualizado: saldo=1200 extrato=200
[ 817ms] ESCRITOR 0 | saiu da região crítica
[ 817ms] ESCRITOR 1 | entrou na região crítica
[ 817ms] ESCRITOR 1 | saque de 50: extrato=150 registrado, saldo=1200 ainda desatualizado
[ 827ms] ESCRITOR 0 | bloqueado: outro escritor ativo
[ 917ms] ESCRITOR 1 | saldo atualizado: saldo=1150 extrato=150
[ 917ms] ESCRITOR 1 | saiu da região crítica
[ 917ms] ESCRITOR 1 | finalizado
[ 918ms] ESCRITOR 0 | entrou na região crítica
[ 918ms] ESCRITOR 0 | depósito de 100: extrato=250 registrado, saldo=1150 ainda desatualizado
[ 1017ms] ESCRITOR 0 | saldo atualizado: saldo=1250 extrato=250
[ 1017ms] ESCRITOR 0 | saiu da região crítica
[ 1017ms] ESCRITOR 0 | finalizado

Final: saldo=1250 extrato=250 | leituras sujas: 8

```

A figura acima mostra o final de uma execução com dois escritores (depósito de 100 e saque de 50). Quando um escritor chega com o outro ativo, ele aparece como bloqueado e só entra depois que o outro sai. Em 313 ms, por exemplo, o extrato já foi atualizado, mas o saldo só muda em 413 ms. É nesse intervalo que ocorrem as leituras sujas. Como os escritores são sincronizados, o resultado final está correto (saldo=1250 e extrato=250), mesmo com **8 leituras sujas** registradas

Exemplo 2

```

[ 2ms] LEITOR 0 | criado
[ 2ms] ESCRITOR 0 | criado (depósito de 25 por rodada)
[ 7ms] LEITOR 0 | entrou na região crítica (sem bloqueio) - consultando saldo
[ 7ms] LEITOR 0 | saldo=1000 extrato=0 (consistente)
[ 7ms] LEITOR 0 | saiu da região crítica
[ 7ms] ESCRITOR 0 | entrou na região crítica
[ 7ms] ESCRITOR 0 | depósito de 25: extrato=25 registrado, saldo=1000 ainda desatualizado
[ 12ms] LEITOR 0 | entrou na região crítica (sem bloqueio) - consultando saldo
[ 12ms] LEITOR 0 | *** LEITURA SUJA *** saldo=1000 extrato=25 (saldo esperado=1025)
[ 12ms] LEITOR 0 | saiu da região crítica
[ 17ms] LEITOR 0 | entrou na região crítica (sem bloqueio) - consultando saldo
[ 17ms] LEITOR 0 | *** LEITURA SUJA *** saldo=1000 extrato=25 (saldo esperado=1025)
[ 17ms] LEITOR 0 | saiu da região crítica
[ 22ms] LEITOR 0 | entrou na região crítica (sem bloqueio) - consultando saldo
[ 22ms] LEITOR 0 | *** LEITURA SUJA *** saldo=1000 extrato=25 (saldo esperado=1025)
[ 22ms] LEITOR 0 | saiu da região crítica
[ 27ms] LEITOR 0 | entrou na região crítica (sem bloqueio) - consultando saldo
[ 28ms] LEITOR 0 | *** LEITURA SUJA *** saldo=1000 extrato=25 (saldo esperado=1025)
[ 28ms] LEITOR 0 | saiu da região crítica
[ 28ms] LEITOR 0 | finalizado
[ 58ms] ESCRITOR 0 | saldo atualizado: saldo=1025 extrato=25
[ 58ms] ESCRITOR 0 | saiu da região crítica
[ 63ms] ESCRITOR 0 | entrou na região crítica
[ 63ms] ESCRITOR 0 | depósito de 25: extrato=50 registrado, saldo=1025 ainda desatualizado
[ 113ms] ESCRITOR 0 | saldo atualizado: saldo=1050 extrato=50
[ 113ms] ESCRITOR 0 | saiu da região crítica
[ 118ms] ESCRITOR 0 | entrou na região crítica
[ 118ms] ESCRITOR 0 | depósito de 25: extrato=75 registrado, saldo=1050 ainda desatualizado
[ 169ms] ESCRITOR 0 | saldo atualizado: saldo=1075 extrato=75
[ 169ms] ESCRITOR 0 | saiu da região crítica
[ 174ms] ESCRITOR 0 | entrou na região crítica
[ 174ms] ESCRITOR 0 | depósito de 25: extrato=100 registrado, saldo=1075 ainda desatualizado
[ 225ms] ESCRITOR 0 | saldo atualizado: saldo=1100 extrato=100
[ 225ms] ESCRITOR 0 | saiu da região crítica
[ 230ms] ESCRITOR 0 | entrou na região crítica
[ 230ms] ESCRITOR 0 | depósito de 25: extrato=125 registrado, saldo=1100 ainda desatualizado
[ 281ms] ESCRITOR 0 | saldo atualizado: saldo=1125 extrato=125
[ 281ms] ESCRITOR 0 | saiu da região crítica
[ 281ms] ESCRITOR 0 | finalizado

Final: saldo=1125 extrato=125 | leituras sujas: 4

```

No exemplo 2, a execução tem um leitor e um escritor, que faz um depósito de 25 por rodada. O leitor consulta o saldo a cada cerca de 5 ms, e cada escrita leva cerca de 50 ms. A primeira leitura (7 ms) é consistente, mas logo depois o escritor registra o extrato=25 e o saldo só muda em 58 ms. Nesse intervalo, o leitor faz mais 4 leituras e todas são sujas: ele vê saldo=1000 e extrato=25, quando o esperado seria 1025. No final, saldo=1125 e extrato=125 estão corretos, e o contador registra **4 leituras sujas**.

- Resultados das Execuções

Comparando os dois exemplos, no exemplo 1 mostra a sincronização entre escritores: com dois escritores, um fica bloqueado enquanto o outro está ativo. No exemplo 2 há só um escritor, então não há bloqueio, e o que aparece com clareza é a leitura suja, já que o leitor nunca espera.

Em ambos o resultado final é correto, porque os escritores são sincronizados, mas as leituras sujas continuam acontecendo, porque os leitores não são. A quantidade delas depende dos tempos: **no exemplo 2, o tempo de escrita é bem maior que o atraso do leitor**, e por isso 4 das 5 leituras foram sujas.

2.2 Escritores com preferência sobre leitores

O tema, as variáveis (saldo e extrato) e a entrada de dados são os mesmos da versão anterior. A diferença é que agora os novos leitores só entram quando não há nenhum escritor ativo ou esperando. Por isso não ocorre leitura suja: o leitor nunca lê no meio de uma atualização.

A sincronização usa três semáforos. O `sem_recurso` dá acesso aos dados: vários leitores podem usá-lo ao mesmo tempo (o primeiro o pega e o último o devolve), enquanto o escritor o pega sozinho. O `sem_leitura` é fechado pelo primeiro escritor que chega, impedindo a entrada de novos leitores, e só é reaberto quando não resta nenhum escritor pendente. É ele que dá a preferência aos escritores. O `sem_fila` deixa só um leitor por vez disputar o `sem_leitura`. Dois semáforos binários (`mutex_l` e `mutex_e`) protegem os contadores de leitores ativos e de escritores pendentes, e um `mutex (io)` serializa a impressão do log. Foram usados semáforos para proteger os dados por ser o mecanismo pedido no enunciado.

Quando um escritor chega, os novos leitores esperam, os leitores que já estavam lendo terminam, e só então o escritor atualiza os dados. Para que a leitura simultânea apareça no log, cada leitor permanece na região crítica por metade do tempo de escrita. A saída mostra, além do que já era exibido na versão anterior, os leitores bloqueados e uma mensagem de desbloqueio sempre que uma thread volta

a executar. Como consequência da preferência, se chegarem escritores continuamente, os leitores podem esperar por muito tempo (inanição).

- Exemplos de Execução

Exemplo 1

```
[ 778ms] ESCRITOR 0 | BLOQUEADO: leitores ou outro escritor usando os dados
[ 869ms] ESCRITOR 1 | saldo atualizado: saldo=1200 extrato=200
[ 869ms] ESCRITOR 1 | saiu da região crítica
[ 869ms] ESCRITOR 0 | desbloqueado
[ 869ms] ESCRITOR 0 | entrou na região crítica
[ 869ms] ESCRITOR 0 | depósito de 100: extrato=300 registrado, saldo=1200 ainda desatualizado
[ 879ms] ESCRITOR 1 | BLOQUEADO: leitores ou outro escritor usando os dados
[ 969ms] ESCRITOR 0 | saldo atualizado: saldo=1300 extrato=300
[ 969ms] ESCRITOR 0 | saiu da região crítica
[ 969ms] ESCRITOR 0 | finalizado
[ 969ms] ESCRITOR 1 | desbloqueado
[ 970ms] ESCRITOR 1 | entrou na região crítica
[ 970ms] ESCRITOR 1 | saque de 50: extrato=250 registrado, saldo=1300 ainda desatualizado
[1069ms] ESCRITOR 1 | saldo atualizado: saldo=1250 extrato=250
[1069ms] ESCRITOR 1 | saiu da região crítica
[1069ms] ESCRITOR 1 | finalizado
[1069ms] LEITOR 1 | desbloqueado
[1069ms] LEITOR 1 | entrou na região crítica (leitores ativos: 1)
[1069ms] LEITOR 0 | desbloqueado
[1069ms] LEITOR 0 | entrou na região crítica (leitores ativos: 2)
[1120ms] LEITOR 0 | saldo=1250 extrato=250 (consistente)
[1120ms] LEITOR 0 | saiu da região crítica
[1120ms] LEITOR 1 | saldo=1250 extrato=250 (consistente)
[1120ms] LEITOR 1 | saiu da região crítica
[1130ms] LEITOR 1 | entrou na região crítica (leitores ativos: 1)
[1130ms] LEITOR 0 | entrou na região crítica (leitores ativos: 2)
[1181ms] LEITOR 1 | saldo=1250 extrato=250 (consistente)
[1181ms] LEITOR 1 | saiu da região crítica
[1181ms] LEITOR 0 | saldo=1250 extrato=250 (consistente)
[1181ms] LEITOR 0 | saiu da região crítica
[1191ms] LEITOR 1 | entrou na região crítica (leitores ativos: 1)
[1192ms] LEITOR 0 | entrou na região crítica (leitores ativos: 2)
[1242ms] LEITOR 1 | saldo=1250 extrato=250 (consistente)
[1242ms] LEITOR 1 | saiu da região crítica
[1242ms] LEITOR 0 | saldo=1250 extrato=250 (consistente)
[1243ms] LEITOR 0 | saiu da região crítica
[1253ms] LEITOR 1 | entrou na região crítica (leitores ativos: 1)
[1253ms] LEITOR 0 | entrou na região crítica (leitores ativos: 2)
[1303ms] LEITOR 1 | saldo=1250 extrato=250 (consistente)
[1304ms] LEITOR 1 | saiu da região crítica
[1304ms] LEITOR 1 | finalizado
[1304ms] LEITOR 0 | saldo=1250 extrato=250 (consistente)
[1304ms] LEITOR 0 | saiu da região crítica
[1304ms] LEITOR 0 | finalizado
Final: saldo=1250 extrato=250 | leituras sujas: 0
```

Os escritores se alternam: o escritor 0 faz um depósito de 100 e o escritor 1 um saque de 50, cada um esperando o outro terminar. Só em 1069 ms, quando os dois escritores terminam, os leitores são desbloqueados e passam a ler juntos. Todas as leituras são consistentes (saldo=1250 e extrato=250), e o contador registra 0 leituras sujas.

Exemplo 2

```
[ 82ms] LEITOR 0 | entrou na região crítica (leitores ativos: 1)
[ 87ms] ESCRITOR 0 | BLOQUEADO: leitores ou outro escritor usando os dados
[ 107ms] LEITOR 0 | saldo=1025 extrato=25 (consistente)
[ 108ms] LEITOR 0 | saiu da região crítica
[ 108ms] ESCRITOR 0 | desbloqueado
[ 108ms] ESCRITOR 0 | entrou na região crítica
[ 108ms] ESCRITOR 0 | depósito de 25: extrato=50 registrado, saldo=1025 ainda desatualizado
[ 113ms] LEITOR 0 | BLOQUEADO: há escritor ativo/esperando
[ 158ms] ESCRITOR 0 | saldo atualizado: saldo=1050 extrato=50
[ 158ms] ESCRITOR 0 | saiu da região crítica
[ 158ms] LEITOR 0 | desbloqueado
[ 159ms] LEITOR 0 | entrou na região crítica (leitores ativos: 1)
[ 164ms] ESCRITOR 0 | BLOQUEADO: leitores ou outro escritor usando os dados
[ 184ms] LEITOR 0 | saldo=1050 extrato=50 (consistente)
[ 184ms] LEITOR 0 | saiu da região crítica
[ 184ms] ESCRITOR 0 | desbloqueado
[ 184ms] ESCRITOR 0 | entrou na região crítica
[ 184ms] ESCRITOR 0 | depósito de 25: extrato=75 registrado, saldo=1050 ainda desatualizado
[ 189ms] LEITOR 0 | BLOQUEADO: há escritor ativo/esperando
[ 235ms] ESCRITOR 0 | saldo atualizado: saldo=1075 extrato=75
[ 235ms] ESCRITOR 0 | saiu da região crítica
[ 235ms] LEITOR 0 | desbloqueado
[ 235ms] LEITOR 0 | entrou na região crítica (leitores ativos: 1)
[ 240ms] ESCRITOR 0 | BLOQUEADO: leitores ou outro escritor usando os dados
[ 261ms] LEITOR 0 | saldo=1075 extrato=75 (consistente)
[ 261ms] LEITOR 0 | saiu da região crítica
[ 261ms] ESCRITOR 0 | desbloqueado
[ 261ms] ESCRITOR 0 | entrou na região crítica
[ 261ms] ESCRITOR 0 | depósito de 25: extrato=100 registrado, saldo=1075 ainda desatualizado
[ 266ms] LEITOR 0 | BLOQUEADO: há escritor ativo/esperando
[ 312ms] ESCRITOR 0 | saldo atualizado: saldo=1100 extrato=100
[ 313ms] ESCRITOR 0 | saiu da região crítica
[ 313ms] LEITOR 0 | desbloqueado
[ 313ms] LEITOR 0 | entrou na região crítica (leitores ativos: 1)
[ 318ms] ESCRITOR 0 | BLOQUEADO: leitores ou outro escritor usando os dados
[ 338ms] LEITOR 0 | saldo=1100 extrato=100 (consistente)
[ 338ms] LEITOR 0 | saiu da região crítica
[ 338ms] LEITOR 0 | finalizado
[ 338ms] ESCRITOR 0 | desbloqueado
[ 338ms] ESCRITOR 0 | entrou na região crítica
[ 338ms] ESCRITOR 0 | depósito de 25: extrato=125 registrado, saldo=1100 ainda desatualizado
[ 389ms] ESCRITOR 0 | saldo atualizado: saldo=1125 extrato=125
[ 389ms] ESCRITOR 0 | saiu da região crítica
[ 389ms] ESCRITOR 0 | finalizado
```

Final: saldo=1125 extrato=125 | leituras sujas: 0

No exemplo 2, há um leitor e um escritor (depósito de 25 por rodada), e os dois se alternam. Quando o escritor está atualizando os dados, o leitor fica bloqueado (por exemplo, em 113 ms), e quando o leitor está lendo, o escritor fica bloqueado (por exemplo, em 164 ms). Cada um só entra depois que o outro sai. Todas as leituras são consistentes, e a execução termina em 389 ms com saldo=1125, extrato=125 e 0 leituras sujas.

- Resultados das Execuções

Comparando os dois exemplos, no exemplo 1 há dois escritores e dois leitores: os leitores esperam até que os dois escritores terminem e depois leem juntos. No exemplo 2, com um escritor e um leitor, os dois se alternam a cada rodada. Nos dois casos o leitor nunca lê durante uma atualização, e por isso **não há nenhuma leitura suja**.

O exemplo 1 também leva mais tempo (1304 ms contra 389 ms), mas isso vem da configuração, e não de um comportamento diferente do programa. No exemplo 1, os

escretores fazem 10 escritas de cerca de 100 ms, uma de cada vez, o que dá cerca de 1 segundo antes de os leitores começarem. No exemplo 2, são apenas 5 escritas de cerca de 50 ms, intercaladas com as leituras.

2.3 Versão sem controle de concorrência

Agora existe apenas a variável saldo. Para conferir os resultados, o programa guarda também a variável esperado, que soma o valor correto de todas as operações e serve apenas de comparação.

Nesta versão não há semáforo nem mutex sobre os dados. O escritor lê o saldo, espera o tempo de escrita e grava o valor lido mais a operação. Se outro escritor alterar o saldo nesse intervalo, a gravação sobrescreve essa alteração, e a operação do outro se perde. Isso é uma condição de corrida (atualização perdida), e o programa conta quantas ocorreram. O problema acontece apenas entre escritores: com um único escritor não há perda. Os leitores só consultam o saldo e o comparam com o esperado, e contam como leitura com valor errado quando os dois são diferentes, o que só acontece depois que uma atualização é perdida.

No fim, o programa mostra o saldo final, o saldo correto, o total de atualizações perdidas e o de leituras com valor errado.

Existem 3 resultados possíveis quando o código é executado, conforme a relação entre o tempo de escrita e o atraso dos escritores:

1. Tempo de escrita grande em relação ao atraso dos escritores: as operações dos escritores se sobrepõem quase sempre, e há muitas atualizações perdidas.
2. Tempo de escrita parecido com o atraso dos escritores: só algumas operações se sobrepõem, e o número de atualizações perdidas varia a cada execução.
3. Tempo de escrita pequeno em relação ao atraso dos escritores: as operações quase não se sobrepõem, e o saldo final pode sair correto, mas o erro continua possível, porque não há nenhuma sincronização.

- Exemplos de Execução

Exemplo 1

```
[ 1ms] LEITOR 0 | criado
[ 7ms] LEITOR 1 | criado
[10ms] ESCRITOR 0 | criado (depósito de 100 por rodada)
[11ms] ESCRITOR 1 | criado (saque de 50 por rodada)
[11ms] LEITOR 0 | consultou saldo=1000 (correto)
[17ms] LEITOR 1 | consultou saldo=1000 (correto)
[21ms] ESCRITOR 0 | leu saldo=1000, vai fazer depósito de 100
[21ms] ESCRITOR 1 | leu saldo=1000, vai fazer saque de 50
[21ms] LEITOR 0 | consultou saldo=1000 (correto)
[27ms] LEITOR 1 | consultou saldo=1000 (correto)
[31ms] LEITOR 0 | consultou saldo=1000 (correto)
[37ms] LEITOR 1 | consultou saldo=1000 (correto)
[41ms] LEITOR 0 | consultou saldo=1000 (correto)
[48ms] LEITOR 1 | consultou saldo=1000 (correto)
[52ms] LEITOR 0 | consultou saldo=1000 (correto)
[52ms] LEITOR 0 | finalizado
[58ms] LEITOR 1 | consultou saldo=1000 (correto)
[58ms] LEITOR 1 | finalizado
[122ms] ESCRITOR 0 | gravou saldo=1100 (correto)
[122ms] ESCRITOR 1 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 1000 para 1100 durante a operação; vai sobrescrever com 950
[122ms] ESCRITOR 1 | *** ERRO *** gravou saldo=950, mas o correto seria 1050 (diferença -100)
[132ms] ESCRITOR 0 | leu saldo=950, vai fazer depósito de 100
[132ms] ESCRITOR 1 | leu saldo=950, vai fazer saque de 50
[232ms] ESCRITOR 0 | *** ERRO *** gravou saldo=1050, mas o correto seria 1150 (diferença -100)
[232ms] ESCRITOR 1 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 950 para 1050 durante a operação; vai sobrescrever com 900
[232ms] ESCRITOR 1 | *** ERRO *** gravou saldo=900, mas o correto seria 1100 (diferença -200)
[243ms] ESCRITOR 1 | leu saldo=900, vai fazer saque de 50
[243ms] ESCRITOR 0 | leu saldo=900, vai fazer depósito de 100
[343ms] ESCRITOR 1 | *** ERRO *** gravou saldo=850, mas o correto seria 1050 (diferença -200)
[343ms] ESCRITOR 0 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 900 para 850 durante a operação; vai sobrescrever com 1000
[343ms] ESCRITOR 0 | *** ERRO *** gravou saldo=1000, mas o correto seria 1150 (diferença -150)
[353ms] ESCRITOR 1 | leu saldo=1000, vai fazer saque de 50
[354ms] ESCRITOR 0 | leu saldo=1000, vai fazer depósito de 100
[454ms] ESCRITOR 1 | *** ERRO *** gravou saldo=950, mas o correto seria 1100 (diferença -150)
[454ms] ESCRITOR 0 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 1000 para 950 durante a operação; vai sobrescrever com 1100
[454ms] ESCRITOR 0 | *** ERRO *** gravou saldo=1100, mas o correto seria 1200 (diferença -100)
[464ms] ESCRITOR 0 | leu saldo=1100, vai fazer depósito de 100
[464ms] ESCRITOR 1 | leu saldo=1100, vai fazer saque de 50
[564ms] ESCRITOR 0 | *** ERRO *** gravou saldo=1200, mas o correto seria 1300 (diferença -100)
[565ms] ESCRITOR 0 | finalizado
[565ms] ESCRITOR 1 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 1100 para 1200 durante a operação; vai sobrescrever com 1050
[565ms] ESCRITOR 1 | *** ERRO *** gravou saldo=1050, mas o correto seria 1250 (diferença -200)
[565ms] ESCRITOR 1 | finalizado

Final: saldo=1050 | saldo correto=1250 | *** SALDO FINAL INCORRETO ***
Atualizações perdidas: 5 | leituras com valor errado: 0
```

No exemplo acima, os dois escritores leem o saldo=1000 ao mesmo tempo e gravam em seguida, e a gravação de um apaga a do outro. Isso aconteceu nas 5 rodadas (5 atualizações perdidas). O saldo final ficou em 1050, 200 a menos que o correto (1250). Os leitores terminaram antes da primeira gravação, então não leram nenhum valor errado.

Exemplo 2

```
[ 0ms] LEITOR 0 | criado
[ 0ms] LEITOR 1 | criado
[ 5ms] LEITOR 0 | consultou saldo=1000 (correto)
[ 5ms] LEITOR 1 | consultou saldo=1000 (correto)
[ 8ms] ESCRITOR 0 | criado (depósito de 50 por rodada)
[ 8ms] ESCRITOR 1 | criado (saque de 25 por rodada)
[10ms] LEITOR 0 | consultou saldo=1000 (correto)
[10ms] LEITOR 1 | consultou saldo=1000 (correto)
[13ms] ESCRITOR 0 | leu saldo=1000, vai fazer depósito de 50
[13ms] ESCRITOR 1 | leu saldo=1000, vai fazer saque de 25
[16ms] LEITOR 0 | consultou saldo=1000 (correto)
[16ms] LEITOR 1 | consultou saldo=1000 (correto)
[21ms] LEITOR 0 | consultou saldo=1000 (correto)
[21ms] LEITOR 1 | consultou saldo=1000 (correto)
[26ms] LEITOR 0 | consultou saldo=1000 (correto)
[26ms] LEITOR 0 | finalizado
[26ms] LEITOR 1 | consultou saldo=1000 (correto)
[26ms] LEITOR 1 | finalizado
[63ms] ESCRITOR 0 | gravou saldo=1050 (correto)
[64ms] ESCRITOR 1 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 1000 para 1050 durante a operação; vai sobrescrever com 975
[64ms] ESCRITOR 1 | *** ERRO *** gravou saldo=975, mas o correto seria 1025 (diferença -50)
[69ms] ESCRITOR 0 | leu saldo=975, vai fazer depósito de 50
[69ms] ESCRITOR 1 | leu saldo=975, vai fazer saque de 25
[119ms] ESCRITOR 0 | *** ERRO *** gravou saldo=1025, mas o correto seria 1075 (diferença -50)
[119ms] ESCRITOR 1 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 975 para 1025 durante a operação; vai sobrescrever com 950
[119ms] ESCRITOR 1 | *** ERRO *** gravou saldo=950, mas o correto seria 1050 (diferença -100)
[124ms] ESCRITOR 0 | leu saldo=950, vai fazer depósito de 50
[124ms] ESCRITOR 1 | leu saldo=950, vai fazer saque de 25
[176ms] ESCRITOR 0 | *** ERRO *** gravou saldo=1000, mas o correto seria 1100 (diferença -100)
[176ms] ESCRITOR 1 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 950 para 1000 durante a operação; vai sobrescrever com 925
[176ms] ESCRITOR 1 | *** ERRO *** gravou saldo=925, mas o correto seria 1075 (diferença -150)
[181ms] ESCRITOR 0 | leu saldo=925, vai fazer depósito de 50
[181ms] ESCRITOR 1 | leu saldo=925, vai fazer saque de 25
[232ms] ESCRITOR 0 | *** ERRO *** gravou saldo=975, mas o correto seria 1125 (diferença -150)
[232ms] ESCRITOR 1 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 925 para 975 durante a operação; vai sobrescrever com 900
[232ms] ESCRITOR 1 | *** ERRO *** gravou saldo=900, mas o correto seria 1100 (diferença -200)
[237ms] ESCRITOR 0 | leu saldo=900, vai fazer depósito de 50
[237ms] ESCRITOR 1 | leu saldo=900, vai fazer saque de 25
[287ms] ESCRITOR 0 | *** ERRO *** gravou saldo=950, mas o correto seria 1150 (diferença -200)
[287ms] ESCRITOR 0 | finalizado
[288ms] ESCRITOR 1 | *** ATUALIZAÇÃO PERDIDA *** saldo mudou de 900 para 950 durante a operação; vai sobrescrever com 875
[288ms] ESCRITOR 1 | *** ERRO *** gravou saldo=875, mas o correto seria 1125 (diferença -250)
[288ms] ESCRITOR 1 | finalizado

Final: saldo=875 | saldo correto=1125 | *** SALDO FINAL INCORRETO ***
Atualizações perdidas: 5 | leituras com valor errado: 0
```

No exemplo 2, os escritores (depósito de 50 e saque de 25) leem o saldo=1000 ao mesmo tempo (13 ms) e, a cada rodada, a gravação de um apaga a do outro: o escritor 0 grava 1050 em 63 ms, e o escritor 1 grava 975 logo depois. Foram 5 atualizações perdidas, como no exemplo 1. O saldo final ficou em 875, 250 a menos que o correto (1125).

- Resultados das Execuções

Os dois exemplos se enquadram no caso 1, em que o tempo de escrita é grande em relação ao atraso dos escritores. Em ambos, a escrita dura cerca de 10 vezes o atraso (100 ms contra 10 ms no exemplo 1, e 50 ms contra 5 ms no exemplo 2). Por isso os dois escritores sempre se sobrepõem, e houve uma atualização perdida em cada uma das 5 rodadas. O saldo final ficou errado nos dois, com erro de 200 no exemplo 1 e de 250 no exemplo 2, porque os valores das operações são diferentes. O exemplo 2 também termina antes (288 ms contra 565 ms), pois a escrita é mais curta.

3. PROCESSOS PRODUTORES X CONSUMIDORES

Para a realização da segunda parte do trabalho, foi utilizado como referência o livro “Sistemas Operacionais: Conceitos e Mecanismos” de Carlos A. Maziero, com foco no Capítulo 8, que introduz os conceitos de comunicação entre tarefas e os possíveis riscos de perda de dados, integridade e ordem (Cap. 8.3.5); o Capítulo 9, que apresenta meios de comunicação e a criação da memória compartilhada; o Capítulo 10, que descreve o conceito de seções críticas e mecanismos para garantir a exclusão mútua (Cap. 10.2, 10.2.2, 10.2.3, 10.2.5); e, por fim, o Capítulo 11, que visa apresentar propriamente os semáforos e *mutexes*, implementados no Trabalho 2 por conseguirem resolver as condições de disputa, presentes no capítulo anterior. Além disso, algumas video aulas foram vistas para auxiliar na implementação do código, bem como o uso do Claude.

Apesar dos sistemas Windows oferecerem chamadas de sistema em C para gerenciar *mutexes*, como `CreateMutex`, `ReleaseMutex`, etc, foi escolhido implementar as funções em Linux, dado que os algoritmos apresentados no livro e nas videoaulas se assemelham a elas.

Vídeos de referência:

▶ 15.3 - Produtor/Consumidor (Bounded Buffer) [SO UFAM]

▶ 15.4 - Produtor/Consumidor e Broadcast [SO UFAM]

▶ 16.3 - Produtor Consumidor com semáforos [SO EUFAM]

▶ Me Salva Sistemas Operacionais: Semáforos com Múltiplos Processos e Com...

▶ Me Salva Sistemas Operacionais: Semáforos - Produtor/Consumidor

3.1 Vários processos produtores e 1 consumidor

Na primeira versão, várias threads produtoras (P) colocam itens em ordem no buffer e uma única thread consumidora (C) retira os itens também em ordem. A sincronização usa três semáforos: *vazias*, que conta as posições livres e começa com o tamanho do buffer; *cheias*, que conta as posições ocupadas e começa em 0; e *mutex*, um semáforo binário que começa em 1 e garante que só uma thread por vez altere o buffer. A escolha de um *mutex* binário provém das implementações vistas no livro, mas também poderia ser utilizado um *mutex* que aceita valores inteiros iguais ou maiores que 0.

Existem 3 resultados possíveis quando o código é executado:

1. Consumidor mais rápido que Produtor

Nesse caso, o buffer fica vazio a maior parte do tempo, dado que os itens adicionados pelo produtor chegam ao buffer e logo são consumidos. Por esse motivo, os Consumidores aparecem com frequência *sleeping*, enquanto os Produtores raramente dormem, pois sempre estão adicionando itens para preencher as posições *vazias*.

2. Consumidor e Produtor com velocidades parecidas

Nesse caso, o buffer oscila bastante entre vazio e cheio. Os resultados são variados, onde Produtores e Consumidores dormem com frequências irregulares.

3. Consumidor mais lento que Produtor

Ao contrário do resultado 1, o buffer fica cheio a maior parte do tempo, dado que os itens adicionados pelo produtor chegam ao buffer e são consumidos em menor velocidade. Por esse motivo, os Produtores aparecem com frequência *sleeping*, enquanto os Consumidores raramente dormem, pois sempre estão removendo itens para esvaziar as posições *cheias*.

- Exemplos de Execução

Exemplo 1

```
[ 4998 ms] Consumidor 1 retirou item 3003 do buffer
[ 4998 ms]   BUFFER: [ 2005 | 3004 | 1004 | --- | 1003 ] ocupados=4
[ 4998 ms] Consumidor 1 consumindo item 3003
[ 4998 ms] Produtor 2 colocou item 2006 no buffer
[ 4998 ms]   BUFFER: [ 2005 | 3004 | 1004 | 2006 | 1003 ] ocupados=5
[ 5510 ms] Consumidor 1 retirou item 1003 do buffer
[ 5511 ms]   BUFFER: [ 2005 | 3004 | 1004 | 2006 | --- ] ocupados=4
[ 5511 ms] Consumidor 1 consumindo item 1003
[ 5511 ms] Produtor 3 colocou item 3005 no buffer
[ 5511 ms]   BUFFER: [ 2005 | 3004 | 1004 | 2006 | 3005 ] ocupados=5
[ 5511 ms] Produtor 3 produzindo item 3006
[ 5691 ms] Produtor 3 dormindo
[ 6211 ms] Consumidor 1 retirou item 2005 do buffer
[ 6211 ms]   BUFFER: [ --- | 3004 | 1004 | 2006 | 3005 ] ocupados=4
[ 6211 ms] Consumidor 1 consumindo item 2005
[ 6211 ms] Produtor 1 colocou item 1005 no buffer
[ 6212 ms]   BUFFER: [ 1005 | 3004 | 1004 | 2006 | 3005 ] ocupados=5
[ 6212 ms] Produtor 1 produzindo item 1006
[ 6349 ms] Produtor 1 dormindo
[ 6521 ms] Consumidor 1 retirou item 3004 do buffer
[ 6521 ms]   BUFFER: [ 1005 | --- | 1004 | 2006 | 3005 ] ocupados=4
[ 6521 ms] Consumidor 1 consumindo item 3004
[ 6521 ms] Produtor 3 colocou item 3006 no buffer
[ 6521 ms]   BUFFER: [ 1005 | 3006 | 1004 | 2006 | 3005 ] ocupados=5
[ 7152 ms] Consumidor 1 retirou item 1004 do buffer
[ 7152 ms]   BUFFER: [ 1005 | 3006 | --- | 2006 | 3005 ] ocupados=4
[ 7152 ms] Consumidor 1 consumindo item 1004
[ 7153 ms] Produtor 1 colocou item 1006 no buffer
[ 7153 ms]   BUFFER: [ 1005 | 3006 | 1006 | 2006 | 3005 ] ocupados=5
[ 8038 ms] Consumidor 1 retirou item 2006 do buffer
[ 8038 ms]   BUFFER: [ 1005 | 3006 | 1006 | --- | 3005 ] ocupados=4
[ 8038 ms] Consumidor 1 consumindo item 2006
[ 8896 ms] Consumidor 1 retirou item 3005 do buffer
[ 8896 ms]   BUFFER: [ 1005 | 3006 | 1006 | --- | --- ] ocupados=3
[ 8897 ms] Consumidor 1 consumindo item 3005
[ 9577 ms] Consumidor 1 retirou item 1005 do buffer
[ 9577 ms]   BUFFER: [ --- | 3006 | 1006 | --- | --- ] ocupados=2
[ 9577 ms] Consumidor 1 consumindo item 1005
[ 10264 ms] Consumidor 1 retirou item 3006 do buffer
[ 10264 ms]   BUFFER: [ --- | --- | 1006 | --- | --- ] ocupados=1
[ 10264 ms] Consumidor 1 consumindo item 3006
[ 10725 ms] Consumidor 1 retirou item 1006 do buffer
[ 10726 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=0
[ 10726 ms] Consumidor 1 consumindo item 1006
```

Fim: todos os 18 itens foram produzidos e consumidos.

Exemplo 2

```
[ 5398 ms] Consumidor 1 retirou item 1004 do buffer
[ 5398 ms]   BUFFER: [ 2004 | 1005 | 3004 | --- | 3003 ]   ocupados=4
[ 5398 ms] Consumidor 1 consumindo item 1004
[ 5398 ms] Produtor 2 colocou item 2005 no buffer
[ 5398 ms]   BUFFER: [ 2004 | 1005 | 3004 | 2005 | 3003 ]   ocupados=5
[ 5398 ms] Produtor 2 produzindo item 2006
[ 5734 ms] Consumidor 1 retirou item 3003 do buffer
[ 5734 ms]   BUFFER: [ 2004 | 1005 | 3004 | 2005 | --- ]   ocupados=4
[ 5734 ms] Consumidor 1 consumindo item 3003
[ 5735 ms] Produtor 1 colocou item 1006 no buffer
[ 5735 ms]   BUFFER: [ 2004 | 1005 | 3004 | 2005 | 1006 ]   ocupados=5
[ 5822 ms] Produtor 2 dormindo
[ 6123 ms] Consumidor 1 retirou item 2004 do buffer
[ 6123 ms]   BUFFER: [ --- | 1005 | 3004 | 2005 | 1006 ]   ocupados=4
[ 6123 ms] Consumidor 1 consumindo item 2004
[ 6123 ms] Produtor 3 colocou item 3005 no buffer
[ 6123 ms]   BUFFER: [ 3005 | 1005 | 3004 | 2005 | 1006 ]   ocupados=5
[ 6123 ms] Produtor 3 produzindo item 3006
[ 6296 ms] Produtor 3 dormindo
[ 6581 ms] Consumidor 1 retirou item 1005 do buffer
[ 6582 ms]   BUFFER: [ 3005 | --- | 3004 | 2005 | 1006 ]   ocupados=4
[ 6582 ms] Consumidor 1 consumindo item 1005
[ 6582 ms] Produtor 2 colocou item 2006 no buffer
[ 6582 ms]   BUFFER: [ 3005 | 2006 | 3004 | 2005 | 1006 ]   ocupados=5
[ 6887 ms] Consumidor 1 retirou item 3004 do buffer
[ 6888 ms]   BUFFER: [ 3005 | 2006 | --- | 2005 | 1006 ]   ocupados=4
[ 6888 ms] Consumidor 1 consumindo item 3004
[ 6888 ms] Produtor 3 colocou item 3006 no buffer
[ 6888 ms]   BUFFER: [ 3005 | 2006 | 3006 | 2005 | 1006 ]   ocupados=5
[ 7447 ms] Consumidor 1 retirou item 2005 do buffer
[ 7447 ms]   BUFFER: [ 3005 | 2006 | 3006 | --- | 1006 ]   ocupados=4
[ 7447 ms] Consumidor 1 consumindo item 2005
[ 7970 ms] Consumidor 1 retirou item 1006 do buffer
[ 7970 ms]   BUFFER: [ 3005 | 2006 | 3006 | --- | --- ]   ocupados=3
[ 7970 ms] Consumidor 1 consumindo item 1006
[ 8279 ms] Consumidor 1 retirou item 3005 do buffer
[ 8279 ms]   BUFFER: [ --- | 2006 | 3006 | --- | --- ]   ocupados=2
[ 8279 ms] Consumidor 1 consumindo item 3005
[ 8800 ms] Consumidor 1 retirou item 2006 do buffer
[ 8800 ms]   BUFFER: [ --- | --- | 3006 | --- | --- ]   ocupados=1
[ 8800 ms] Consumidor 1 consumindo item 2006
[ 9618 ms] Consumidor 1 retirou item 3006 do buffer
[ 9618 ms]   BUFFER: [ --- | --- | --- | --- | --- ]   ocupados=0
[ 9618 ms] Consumidor 1 consumindo item 3006

Fim: todos os 18 itens foram produzidos e consumidos.
```

(Casos de tipo de resultado 3)

3.2 Vários processos produtores e consumidores

Na segunda versão, várias threads produtoras (P) colocam itens em ordem no buffer e várias threads consumidoras (C) retiram os itens também em ordem. Os semáforos são os mesmos da versão anterior (*vazias*, *cheias* e *mutex*), mas agora o *mutex* protege tanto a escrita quanto a leitura, porque os Consumidores também disputam o buffer.

Existem 3 resultados possíveis quando o código é executado, semelhantes aos casos apresentados na versão 1. Para evitar que o relatório fique redundante, os casos possíveis serão apresentados apenas na seção 3.1.

- Exemplos de Execução

Exemplo 1

```
[ 2366 ms] Produtor 2 dormindo (buffer cheio)
[ 2367 ms] Consumidor 1 retirou item 3003 do buffer
[ 2367 ms]   BUFFER: [ 1003 | 3005 | 2005 | --- | 3004 ] ocupados=4
[ 2367 ms] Consumidor 1 consumindo item 3003
[ 2367 ms] Produtor 1 colocou item 1004 no buffer
[ 2367 ms]   BUFFER: [ 1003 | 3005 | 2005 | 1004 | 3004 ] ocupados=5
[ 2367 ms] Produtor 1 produzindo item 1005
[ 2536 ms] Consumidor 2 retirou item 3004 do buffer
[ 2536 ms]   BUFFER: [ 1003 | 3005 | 2005 | 1004 | --- ] ocupados=4
[ 2536 ms] Consumidor 2 consumindo item 3004
[ 2536 ms] Produtor 3 colocou item 3006 no buffer
[ 2536 ms]   BUFFER: [ 1003 | 3005 | 2005 | 1004 | 3006 ] ocupados=5
[ 2955 ms] Consumidor 2 retirou item 1003 do buffer
[ 2955 ms]   BUFFER: [ --- | 3005 | 2005 | 1004 | 3006 ] ocupados=4
[ 2955 ms] Consumidor 2 consumindo item 1003
[ 2955 ms] Produtor 2 colocou item 2006 no buffer
[ 2956 ms]   BUFFER: [ 2006 | 3005 | 2005 | 1004 | 3006 ] ocupados=5
[ 2959 ms] Produtor 1 dormindo (buffer cheio)
[ 3250 ms] Consumidor 1 retirou item 3005 do buffer
[ 3250 ms]   BUFFER: [ 2006 | --- | 2005 | 1004 | 3006 ] ocupados=4
[ 3250 ms] Consumidor 1 consumindo item 3005
[ 3250 ms] Produtor 1 colocou item 1005 no buffer
[ 3250 ms]   BUFFER: [ 2006 | 1005 | 2005 | 1004 | 3006 ] ocupados=5
[ 3250 ms] Produtor 1 produzindo item 1006
[ 3628 ms] Consumidor 2 retirou item 2005 do buffer
[ 3628 ms]   BUFFER: [ 2006 | 1005 | --- | 1004 | 3006 ] ocupados=4
[ 3628 ms] Consumidor 2 consumindo item 2005
[ 3709 ms] Produtor 1 colocou item 1006 no buffer
[ 3709 ms]   BUFFER: [ 2006 | 1005 | 1006 | 1004 | 3006 ] ocupados=5
[ 4111 ms] Consumidor 1 retirou item 1004 do buffer
[ 4111 ms]   BUFFER: [ 2006 | 1005 | 1006 | --- | 3006 ] ocupados=4
[ 4111 ms] Consumidor 1 consumindo item 1004
[ 4370 ms] Consumidor 2 retirou item 3006 do buffer
[ 4370 ms]   BUFFER: [ 2006 | 1005 | 1006 | --- | --- ] ocupados=3
[ 4370 ms] Consumidor 2 consumindo item 3006
[ 4522 ms] Consumidor 1 retirou item 2006 do buffer
[ 4522 ms]   BUFFER: [ --- | 1005 | 1006 | --- | --- ] ocupados=2
[ 4522 ms] Consumidor 1 consumindo item 2006
[ 5061 ms] Consumidor 1 retirou item 1005 do buffer
[ 5061 ms]   BUFFER: [ --- | --- | 1006 | --- | --- ] ocupados=1
[ 5061 ms] Consumidor 1 consumindo item 1005
[ 5076 ms] Consumidor 2 retirou item 1006 do buffer
[ 5076 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=0
[ 5076 ms] Consumidor 2 consumindo item 1006
```

Fim: todos os 18 itens foram produzidos e consumidos.

Exemplo 2

```
[ 2392 ms] Consumidor 2 retirou item 1003 do buffer
[ 2392 ms]   BUFFER: [ 3005 | 1004 | 2004 | --- | 2003 ] ocupados=4
[ 2392 ms] Consumidor 2 consumindo item 1003
[ 2392 ms] Produtor 1 colocou item 1005 no buffer
[ 2392 ms]   BUFFER: [ 3005 | 1004 | 2004 | 1005 | 2003 ] ocupados=5
[ 2392 ms] Produtor 1 produzindo item 1006
[ 2535 ms] Consumidor 1 retirou item 2003 do buffer
[ 2535 ms]   BUFFER: [ 3005 | 1004 | 2004 | 1005 | --- ] ocupados=4
[ 2535 ms] Consumidor 1 consumindo item 2003
[ 2535 ms] Produtor 3 colocou item 3006 no buffer
[ 2535 ms]   BUFFER: [ 3005 | 1004 | 2004 | 1005 | 3006 ] ocupados=5
[ 2730 ms] Produtor 1 dormindo (buffer cheio)
[ 3165 ms] Consumidor 2 retirou item 3005 do buffer
[ 3165 ms]   BUFFER: [ --- | 1004 | 2004 | 1005 | 3006 ] ocupados=4
[ 3165 ms] Consumidor 2 consumindo item 3005
[ 3165 ms] Produtor 2 colocou item 2005 no buffer
[ 3165 ms]   BUFFER: [ 2005 | 1004 | 2004 | 1005 | 3006 ] ocupados=5
[ 3165 ms] Produtor 2 produzindo item 2006
[ 3298 ms] Consumidor 1 retirou item 1004 do buffer
[ 3298 ms]   BUFFER: [ 2005 | --- | 2004 | 1005 | 3006 ] ocupados=4
[ 3298 ms] Consumidor 1 consumindo item 1004
[ 3298 ms] Produtor 1 colocou item 1006 no buffer
[ 3298 ms]   BUFFER: [ 2005 | 1006 | 2004 | 1005 | 3006 ] ocupados=5
[ 3479 ms] Produtor 2 dormindo (buffer cheio)
[ 3872 ms] Consumidor 2 retirou item 2004 do buffer
[ 3872 ms]   BUFFER: [ 2005 | 1006 | --- | 1005 | 3006 ] ocupados=4
[ 3872 ms] Consumidor 2 consumindo item 2004
[ 3872 ms] Produtor 2 colocou item 2006 no buffer
[ 3872 ms]   BUFFER: [ 2005 | 1006 | 2006 | 1005 | 3006 ] ocupados=5
[ 4035 ms] Consumidor 1 retirou item 1005 do buffer
[ 4035 ms]   BUFFER: [ 2005 | 1006 | 2006 | --- | 3006 ] ocupados=4
[ 4036 ms] Consumidor 1 consumindo item 1005
[ 4462 ms] Consumidor 2 retirou item 3006 do buffer
[ 4462 ms]   BUFFER: [ 2005 | 1006 | 2006 | --- | --- ] ocupados=3
[ 4462 ms] Consumidor 2 consumindo item 3006
[ 4761 ms] Consumidor 1 retirou item 2005 do buffer
[ 4761 ms]   BUFFER: [ --- | 1006 | 2006 | --- | --- ] ocupados=2
[ 4761 ms] Consumidor 1 consumindo item 2005
[ 4965 ms] Consumidor 2 retirou item 1006 do buffer
[ 4965 ms]   BUFFER: [ --- | --- | 2006 | --- | --- ] ocupados=1
[ 4965 ms] Consumidor 2 consumindo item 1006
[ 5653 ms] Consumidor 1 retirou item 2006 do buffer
[ 5653 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=0
[ 5653 ms] Consumidor 1 consumindo item 2006
```

Fim: todos os 18 itens foram produzidos e consumidos.

3.3 Versão sem controle de concorrência

Na terceira versão, o problema é o mesmo das anteriores, mas sem nenhum semáforo protegendo o buffer, os índices de escrita e leitura e o contador ocupados. Nenhuma thread espera pela outra: os Produtores escrevem mesmo com o buffer cheio e os Consumidores leem mesmo com o buffer vazio. Por isso, nenhuma thread aparece como dormindo nessa versão. Para deixar o problema visível, há uma “pausa” (`janela_ms`) entre ler e atualizar cada variável compartilhada. Sem ela o erro continua existindo, mas aparece de forma mais rara.

- Exemplos de Execução

Exemplo 1

```
[ 2007 ms] Consumidor 1 retirou item 3005 da posicao 0
[ 2007 ms]   BUFFER: [ --- | 2005 | 2004 | 3004 | --- ] ocupados=5
[ 2007 ms] Consumidor 1 consumindo item 3005
[ 2095 ms] Produtor 1: posicao 2 ja tem o item 2004, sera sobrescrito (perdido)
[ 2116 ms] Produtor 1 colocou item 1005 na posicao 2
[ 2116 ms]   BUFFER: [ --- | 2005 | 1005 | 3004 | --- ] ocupados=6
[ 2116 ms] Produtor 1 produzindo item 1006
[ 2419 ms] Produtor 3: posicao 3 ja tem o item 3004, sera sobrescrito (perdido)
[ 2428 ms] Consumidor 2 retirou item 2005 da posicao 1
[ 2428 ms]   BUFFER: [ --- | --- | 1005 | 3006 | --- ] ocupados=5
[ 2428 ms] Consumidor 2 consumindo item 2005
[ 2439 ms] Produtor 3 colocou item 3006 na posicao 3
[ 2439 ms]   BUFFER: [ --- | --- | 1005 | 3006 | --- ] ocupados=7
[ 2481 ms] Produtor 2 colocou item 2006 na posicao 4
[ 2481 ms]   BUFFER: [ --- | --- | 1005 | 3006 | 2006 ] ocupados=8
[ 2671 ms] Produtor 1 colocou item 1006 na posicao 0
[ 2671 ms]   BUFFER: [ 1006 | --- | 1005 | 3006 | 2006 ] ocupados=9
[ 2774 ms] Consumidor 1 retirou item 1005 da posicao 2
[ 2774 ms]   BUFFER: [ 1006 | --- | --- | 3006 | 2006 ] ocupados=8
[ 2774 ms] Consumidor 1 consumindo item 1005
[ 2834 ms] Consumidor 2 retirou item 3006 da posicao 3
[ 2834 ms]   BUFFER: [ 1006 | --- | --- | --- | 2006 ] ocupados=7
[ 2834 ms] Consumidor 2 consumindo item 3006
[ 3257 ms] Consumidor 2 retirou item 2006 da posicao 4
[ 3258 ms]   BUFFER: [ 1006 | --- | --- | --- | --- ] ocupados=6
[ 3258 ms] Consumidor 2 consumindo item 2006
[ 3621 ms] Consumidor 1 retirou item 1006 da posicao 0
[ 3622 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=5
[ 3622 ms] Consumidor 1 consumindo item 1006
[ 3774 ms] Consumidor 2: posicao 1 esta vazia, nao esperou um item (errado)
[ 3794 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=4
[ 4078 ms] Consumidor 1: posicao 2 esta vazia, nao esperou um item (errado)
[ 4099 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=3
[ 4344 ms] Consumidor 2: posicao 3 esta vazia, nao esperou um item (errado)
[ 4365 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=2
[ 4436 ms] Consumidor 1: posicao 4 esta vazia, nao esperou um item (errado)
[ 4457 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=1
[ 4812 ms] Consumidor 1: posicao 0 esta vazia, nao esperou um item (errado)
[ 4833 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=0
[ 5190 ms] Consumidor 2: posicao 1 esta vazia, nao esperou um item (errado)
[ 5211 ms]   BUFFER: [ --- | --- | --- | --- | --- ] ocupados=-1
Resultados (deveria ser 0 erros e contador = 0)
Itens sobrescritos (perdidos): 8
Leituras de posicao vazia: 8
Itens restantes no buffer: 0
Contador ocupado final: -1 (deveria ser 0)
```

Exemplo 2

```
[ 1818 ms] BUFFER: [ --- | 2004 | 3004 | 1006 | 3003 ] ocupados=8
[ 1902 ms] Consumidor 1 retirou item 2004 da posicao 1
[ 1902 ms] BUFFER: [ --- | --- | 3004 | 1006 | 3003 ] ocupados=7
[ 1903 ms] Consumidor 1 consumindo item 2004
[ 1918 ms] Produtor 2: posicao 4 ja tem o item 3003, sera sobrescrito (perdido)
[ 1939 ms] Produtor 2 colocou item 2005 na posicao 4
[ 1939 ms] BUFFER: [ --- | --- | 3004 | 1006 | 2005 ] ocupados=8
[ 1939 ms] Produtor 2 produzindo item 2006
[ 2164 ms] Produtor 2 colocou item 2006 na posicao 0
[ 2164 ms] BUFFER: [ 2006 | --- | 3004 | 1006 | 2005 ] ocupados=9
[ 2208 ms] Produtor 3 colocou item 3005 na posicao 1
[ 2208 ms] BUFFER: [ 2006 | 3005 | 3004 | 1006 | 2005 ] ocupados=10
[ 2208 ms] Produtor 3 produzindo item 3006
[ 2508 ms] Consumidor 2 retirou item 3004 da posicao 2
[ 2508 ms] BUFFER: [ 2006 | 3005 | --- | 1006 | 2005 ] ocupados=9
[ 2508 ms] Consumidor 2 consumindo item 3004
[ 2629 ms] Produtor 3 colocou item 3006 na posicao 2
[ 2630 ms] BUFFER: [ 2006 | 3005 | 3006 | 1006 | 2005 ] ocupados=10
[ 2771 ms] Consumidor 1 retirou item 1006 da posicao 3
[ 2771 ms] BUFFER: [ 2006 | 3005 | 3006 | --- | 2005 ] ocupados=9
[ 2771 ms] Consumidor 1 consumindo item 1006
[ 3299 ms] Consumidor 1 retirou item 2005 da posicao 4
[ 3299 ms] BUFFER: [ 2006 | 3005 | 3006 | --- | --- ] ocupados=8
[ 3299 ms] Consumidor 1 consumindo item 2005
[ 3375 ms] Consumidor 2 retirou item 2006 da posicao 0
[ 3375 ms] BUFFER: [ --- | 3005 | 3006 | --- | --- ] ocupados=7
[ 3375 ms] Consumidor 2 consumindo item 2006
[ 4209 ms] Consumidor 1 retirou item 3005 da posicao 1
[ 4209 ms] BUFFER: [ --- | --- | 3006 | --- | --- ] ocupados=6
[ 4209 ms] Consumidor 1 consumindo item 3005
[ 4244 ms] Consumidor 2 retirou item 3006 da posicao 2
[ 4244 ms] BUFFER: [ --- | --- | --- | --- | --- ] ocupados=5
[ 4244 ms] Consumidor 2 consumindo item 3006
[ 4613 ms] Consumidor 2: posicao 3 esta vazia, nao esperou um item (errado)
[ 4633 ms] BUFFER: [ --- | --- | --- | --- | --- ] ocupados=4
[ 4741 ms] Consumidor 1: posicao 4 esta vazia, nao esperou um item (errado)
[ 4761 ms] BUFFER: [ --- | --- | --- | --- | --- ] ocupados=3
[ 5152 ms] Consumidor 1: posicao 0 esta vazia, nao esperou um item (errado)
[ 5173 ms] BUFFER: [ --- | --- | --- | --- | --- ] ocupados=2
[ 5391 ms] Consumidor 2: posicao 1 esta vazia, nao esperou um item (errado)
[ 5411 ms] BUFFER: [ --- | --- | --- | --- | --- ] ocupados=1
Resultados (deveria ser 0 erros e contador = 0)
Itens sobrescritos (perdidos): 6
Leituras de posicao vazia: 6
Itens restantes no buffer: 0
Contador ocupado final: 1 (deveria ser 0)
```

4. CONCLUSÃO

Os experimentos mostraram que os semáforos são essenciais para a integridade dos dados compartilhados. Sem controle de concorrência, houve atualizações perdidas em Leitores × Escritores e, em Produtores × Consumidores, itens sobrescritos, leituras de posições vazias e contador inconsistente. Com semáforos, esses erros não ocorreram.

Em Leitores × Escritores, a preferência aos escritores eliminou as leituras sujas, com o risco de inanição dos leitores. Já em Produtores × Consumidores, os semáforos vazias e cheias controlaram a espera por posição livre ou item disponível, e o mutex garantiu o acesso exclusivo ao buffer.

5. FONTES

- O livro de Maziero serviu como fundamentação teórica, e o Claude (Anthropic) auxiliou na implementação dos códigos.

MAZIERO, Carlos A. **Sistemas Operacionais: Conceitos e Mecanismos**. Curitiba: UFPR, 2019. Disponível em: <https://wiki.inf.ufpr.br/maziero/lib/exe/fetch.php?media=socm%3Asocm-livro.pdf>. Acesso em: 9 out. 2026.

ANTHROPIC. **Claude**. Modelo de linguagem. Disponível em: <https://claude.ai>. Acesso em: 9 out. 2026.